HACKATHON – DOCUMENT PARTICIPANTS

AI Banking Copilot for Fraud Detection

on Microsoft Fabric

Un défi centré sur la détection de fraude bancaire, l'explicabilité des décisions et l'assistance à l'analyste via un copilote intelligent.

Objectif	Concevoir une application IA capable de **détecter, expliquer et prioriser** des transactions suspectes, et d'aider l'analyste fraude à **décider**.
Données	Jeu de données bancaire 100 % synthétique : 2 500 clients, ~435 000 transactions, ~192 000 événements de banque digitale (janvier → septembre 2026).
Plateforme data	Microsoft Fabric (Lakehouse, Notebooks)
Copilote métier	Azure OpenAI + interface interactive
Interface recommandée	Streamlit
Assistance au développement	GitHub Copilot

Ce document remplace la version précédente du brief. Il conserve le cadre du challenge et ajoute tout ce qu'il faut pour démarrer vite : description des données, parcours recommandés, idées de features, conception du copilote, architecture Fabric, planning, grille d'évaluation et code de démarrage.

Sommaire

L’essentiel en une page
1. Thème et vision du challenge
2. Contexte et problématique
3. Résultats attendus
4. Stack technologique
5. Les données du hackathon
6. Trois parcours recommandés
7. Feature engineering : par où commencer
8. Détection et scoring
9. Concevoir le copilote
10. L’interface analyste
11. Architecture de référence sur Microsoft Fabric
12. Organisation et planning conseillés
13. Livrables
14. Soumission des scores (optionnelle)
15. Critères d’évaluation
16. Règles du jeu
17. Réussir son pitch
18. Message final
Annexe A – Glossaire
Annexe B – FAQ
Annexe C – Code de démarrage

L’essentiel en une page

> **Votre mission**
>
> Construire un **copilote pour l’analyste fraude** d’une banque de détail tunisienne fictive. Il doit **détecter** les transactions suspectes, **expliquer** pourquoi, **dialoguer** avec l’analyste en langage naturel et l’aider à **décider** : bloquer, escalader ou valider.

Vous recevez	10 fichiers CSV synthétiques : clients, cartes, commerçants, ~435 000 transactions, événements de banque digitale, labels fraude (janvier → juillet), alertes du moteur de règles actuel, dictionnaire de données (`README.md`).
Vous livrez	Une application démontrable (file d’alertes, score, explication, copilote, décision), un pipeline data sur Microsoft Fabric, un dépôt GitHub et un pitch de 5 minutes avec démo live.
Parcours minimal	`transactions_train.csv` + `fraud_labels_train.csv` + `customers.csv` suffisent pour une solution complète. Les autres fichiers sont des bonus qui enrichissent la détection et les explications.
Point de départ	Le moteur de règles actuel de la banque génère **85 % de faux positifs** et laisse passer **58 % des fraudes**. Votre solution doit faire mieux, et le démontrer.
Évaluation	Pertinence métier, qualité de la détection, qualité des explications et du copilote, démo, cohérence technologique (voir § 15).

Trois règles d’or

1. **Le copilote parle à partir des données.** Chaque affirmation doit être vérifiable : montant, date, identifiant, comparaison à l’historique.
2. **Une transaction se juge avec son passé uniquement.** N’utilisez jamais une information postérieure à la transaction analysée.
3. **Rendez visible le chemin transaction → score → explication → décision** en moins de 30 secondes de démo.

1. Ingestion	2. Détection	3. Explication	4. Interaction	5. Décision	6. Amélioration
Données dans Fabric	Features + modèle	Facteurs + Azure OpenAI	Analyste ↔ copilote	Bloquer / escalader / valider	Feedback analyste

*La décision finale appartient toujours à l’analyste : l’IA **augmente** la décision, elle ne la remplace pas.*

1. Thème et vision du challenge

Thème officiel : AI Banking Copilot for Fraud Detection on Microsoft Fabric.

Le défi se situe à l'intersection de trois dimensions : la détection de fraude bancaire, l'assistance métier par l'IA générative et l'industrialisation data sur Microsoft Fabric.

L'objectif n'est **ni un simple moteur de scoring, ni un simple chatbot**. Le résultat attendu est un copilote bancaire orienté analyste fraude, capable d'interpréter les signaux, d'expliquer les anomalies et d'aider à la décision de manière opérationnelle.

1.1 Transformation visée

- Détecter des anomalies ou des signaux faibles à partir de données transactionnelles et comportementales.
- Calculer un score de risque qui permet de prioriser les alertes.
- Expliquer la décision ou les facteurs de suspicion dans une formulation claire et métier.
- Permettre à l'analyste de poser des questions simples, ciblées et utiles.

1.2 Positionnement du copilote

Le copilote ne remplace pas l'analyste. C'est un assistant qui accélère l'investigation, facilite la compréhension et améliore la qualité des arbitrages. Il transforme des données complexes en éléments directement exploitables.

2. Contexte et problématique

La fraude bancaire est aujourd'hui multicanale, rapide, organisée et adaptative : paiements en ligne, opérations mobiles, paiements en magasin, retraits, opérations à l'international, ou comportements atypiques dans des parcours en apparence légitimes.

Enjeu	Ce que cela implique
Risque financier	Pertes directes (fonds détournés, transactions non autorisées), effet d'échelle dans le digital, coût des équipes d'investigation.
Risque réglementaire	KYC, AML, surveillance : la décision doit être **traçable, explicable et justifiable** auprès du régulateur.
Confiance client	Une fraude mal gérée, ou une carte bloquée à tort pendant un voyage, dégrade l'image et accélère le départ des clients.

2.1 Les limites du système actuel

- **Règles statiques** : seuil de montant, pays étranger, fréquence élevée. Faciles à comprendre, elles captent mal les combinaisons de signaux faibles et demandent une maintenance lourde.
- **Trop de faux positifs** : un seul critère dépassé suffit à déclencher une alerte. Les analystes perdent du temps et se fatiguent.
- **Explications pauvres** : l'analyste doit reconstruire lui-même la logique à partir de plusieurs sources.
- **Cycle lent** : détection → revue manuelle → analyse approfondie → décision. L'enjeu est de réduire le temps entre l'alerte et la décision, sans perdre en qualité ni en traçabilité.

Ce que disent les données du hackathon

Le fichier `legacy_rule_alerts_train.csv` contient les **6 324 alertes** levées par le moteur de règles actuel entre janvier et juillet 2026, avec la décision finale de l'analyste :

- **85 %** de ces alertes étaient des **faux positifs** ;
- seules **42 %** des fraudes ont déclenché une alerte ;
- exemple : un client parti dix jours à Istanbul en juillet a généré à lui seul **21 alertes**, toutes injustifiées.

C'est votre point de comparaison : montrez combien d'alertes inutiles votre solution évite et combien de fraudes supplémentaires elle détecte.

3. Résultats attendus

La solution finale doit démontrer quatre capacités complémentaires :

Capacité	Ce que la solution doit faire	Ce que le jury veut voir
Détecter	Identifier les transactions suspectes et leur attribuer un score de risque.	Une file d'alertes priorisée, un score crédible et validé, une comparaison avec le moteur de règles.
Comprendre	Expliquer les facteurs qui ont conduit au score.	Des facteurs chiffrés et comparés à l'historique du client, pas des généralités.
Interagir	Dialoguer avec l'analyste pour accélérer son analyse.	Des réponses en direct à des questions libres, reliées aux données.
Décider	Aider l'analyste à trancher et garder une trace.	Une recommandation (bloquer / escalader / valider), une décision enregistrée, un feedback.

Exemple d'explication attendue : « Cette transaction est suspecte car son montant est 5 fois supérieur aux habitudes du client et elle a lieu dans un pays où il n'a jamais payé physiquement. » La forme peut varier, mais l'explication doit être **claire, chiffrée et orientée décision**.

4. Stack technologique

Composant	Rôle dans la solution	Statut
Microsoft Fabric	Ingestion, stockage (Lakehouse), préparation et features (Notebooks), scoring.	Attendu
GitHub + GitHub Copilot	Versionnement du code et accélération du développement (code, tests, interface, documentation).	Attendu
Azure OpenAI	Génération des explications et dialogue avec l'analyste (function calling recommandé).	Recommandé
Streamlit	Interface analyste : alertes, fiche transaction, vue client, copilote, décision.	Recommandé
Python (pandas, PySpark)	Préparation des données et feature engineering.	Recommandé
scikit-learn, LightGBM, XGBoost, SHAP	Modèle de scoring et explicabilité.	Libre

Vous restez libres de votre implémentation, mais la solution doit montrer **un usage cohérent de Fabric comme plateforme data et de GitHub Copilot comme accélérateur de delivery**.

Contraintes

- Pas de simple dashboard statique.

- Pas de modèle sans interface utilisateur.
- Pas de chatbot générique déconnecté des données du cas fraude.

5. Les données du hackathon

5.1 Le contexte

Les données simulent l'activité d'une **banque de détail tunisienne fictive** du 1er janvier au 30 septembre 2026 : 2 500 clients (étudiants, particuliers, premium, professionnels, retraités) répartis dans 18 villes, avec leurs cartes nationales ou internationales, leurs paiements en magasin et en ligne, leurs retraits, leurs virements et leur usage de l'application mobile.

- Données **100 % synthétiques** : aucun client réel, tous les identifiants et noms de commerçants sont fictifs.
- Montants en **dinars tunisiens (TND)**, avec 3 décimales. Horodatages à l'heure de Tunis.
- Taux de fraude d'environ **0,65 %** : les classes sont fortement déséquilibrées.
- Le dictionnaire de données complet (toutes les colonnes et tous les codes) se trouve dans `README.md`.

5.2 Les fichiers

Fichier	Contenu	Lignes	Priorité
`transactions_train.csv`	Transactions de janvier à juillet 2026	333 059	Indispensable
`fraud_labels_train.csv`	Label fraude (0/1), typologie de fraude, date de signalement	333 059	Indispensable
`customers.csv`	Profil client : âge, segment, revenu, ville, KYC, usage digital	2 500	Indispensable
`transactions_test.csv`	Transactions d'août à septembre 2026, **sans label** (à scorer)	101 730	Indispensable
`merchants.csv`	Commerçants, sites e-commerce et distributeurs (DAB)	2 621	Recommandé
`cards.csv`	Contrats cartes : type, international ou non, plafonds, sans contact	2 940	Recommandé
`legacy_rule_alerts_train.csv`	Alertes du moteur de règles actuel + décision de l'analyste	6 324	Recommandé
`digital_events.csv`	Connexions, échecs, nouvel appareil, ajout de bénéficiaire, mot de passe, numéro de téléphone	192 401	Bonus (clé pour la prise de contrôle de compte)
`sample_submission.csv`	Format de soumission des scores	101 730	Si classement
`README.md`	Dictionnaire de données complet	–	À lire en premier

5.3 Comment les fichiers se relient

Lien	Clé de jointure
Transaction ↔ label / alerte du moteur de règles	`transaction_id`
Transaction ↔ client ↔ carte	`customer_id`, `card_id`
Transaction ↔ commerçant / DAB / site	`merchant_id`
Transaction mobile ou web ↔ événements digitaux de la même session	`session_id`
Appareils, bénéficiaires (communs à plusieurs transactions ou clients)	`device_id`, `beneficiary_id`

5.4 Découpage temporel

Période	Données	Usage conseillé
Janvier → mai 2026	Transactions avec labels	Entraînement du modèle
Juin → juillet 2026	Transactions avec labels	Validation (mesure honnête de la performance)
Août → septembre 2026	Transactions **sans** labels	Scoring final, file d'alertes de la démo, soumission optionnelle

Pour la démo, considérez les transactions d'août et septembre comme le **flux du jour** de l'analyste : vous ne connaissez pas la réponse, comme dans la vraie vie. Les fraudes labellisées de janvier à juillet servent à illustrer et à calibrer vos explications.

5.5 Les codes à connaître

Colonne	Valeurs	À savoir
`channel`	TPE, DAB, EN_LIGNE, MOBILE, WEB	TPE = paiement en magasin, DAB = distributeur. MOBILE et WEB = banque en ligne (avec `session_id`).
`entry_mode`	CHIP, SANS_CONTACT, MAGSTRIPE, ECOM_3DS, ECOM_SANS_3DS, APP_AUTH	La piste magnétique (MAGSTRIPE) est facile à copier. Le paiement sans contact ne demande pas de code PIN sous 200 TND.
`transaction_type`	ACHAT, RETRAIT, VIREMENT, PAIEMENT_FACTURE, RECHARGE	Les virements n'ont pas de carte ni de commerçant, mais un `beneficiary_id`.
`status` / `decline_reason`	APPROUVEE / REFUSEE + motif	Motifs : solde insuffisant, plafond, PIN erroné, CVV invalide, date d'expiration invalide, carte non internationale, carte en opposition. **Les tentatives refusées peuvent être des fraudes.**
`event_type`	CONNEXION, AJOUT_APPAREIL, AJOUT_BENEFICIAIRE, REINITIALISATION_MOT_DE_PASSE, MODIFICATION_NUMERO_TELEPHONE	Avec `status` (SUCCES / ECHEC), `channel` (dont AGENCE et CENTRE_APPEL) et `auth_method`.

Les cellules vides sont normales : pas de carte pour un virement, pas d'appareil pour un paiement en magasin, pas de coordonnées pour un achat en ligne. Le détail est dans `README.md`.

5.6 Les typologies de fraude

Chaque transaction frauduleuse de la période d'entraînement porte une typologie (`fraud_scenario`). Ces typologies sont un excellent support pour vos explications et pour mesurer ce que votre modèle détecte bien ou mal.

Typologie	Ce qui se passe	Où regarder
CARD_TESTING	Des données de carte volées sont testées par de petits paiements en ligne, puis utilisées pour de gros achats.	Rafales de micro-montants, refus en série, appareil ou IP inhabituels, catégories faciles à revendre.
ACCOUNT_TAKEOVER	Un fraudeur prend le contrôle de l'accès à la banque en ligne et vide le compte par virements.	`digital_events` juste avant le virement : échecs de connexion, nouvel appareil, changement de numéro, nouveau bénéficiaire. Montant comparé aux habitudes.

Typologie	Ce qui se passe	Où regarder
CARTE_CLONEE_ETRANGER	Une copie de la carte est utilisée physiquement à l'étranger.	Pays jamais visité en paiement physique, mode d'entrée, activité simultanée du vrai client en Tunisie.
SKIMMING_DAB	Les données de la carte ont été copiées sur un distributeur. La carte est ensuite vidée par des retraits.	Retraits rapprochés, ville inhabituelle, horaires, plafonds atteints.
CARTE_VOLEE_SANS_CONTACT	Une carte physique volée est utilisée sans code PIN.	Paiements sans contact successifs juste sous le plafond, commerçants inconnus du client.
FRAUDE_EN_LIGNE_DISCRETE	Fraude étalée sur plusieurs jours, avec des montants modestes.	Combinaison de signaux faibles : appareils, IP, catégories, rythme. **La plus difficile.**

> **Indice : la fraude est parfois organisée**
>
> Certaines fraudes ne sont pas isolées : plusieurs victimes partagent un **point commun** (un commerçant, un distributeur, un site web, un appareil) avant d'être fraudées. Une analyse « en graphe » de ces points communs est un vrai plus, pour la détection comme pour la démo.

5.7 Les pièges : des comportements légitimes qui ressemblent à de la fraude

Réduire les faux positifs est au cœur du sujet. Votre copilote doit savoir expliquer pourquoi une alerte **n'est pas** une fraude.

Situation légitime	Pourquoi ça ressemble à de la fraude	Ce qui permet de trancher
Voyage à l'étranger (vacances, Omra, affaires)	Pays nouveau, montants élevés, piste magnétique sur des terminaux anciens.	Billet ou hôtel réservé avant, plus d'activité physique en Tunisie pendant le séjour, IP du pays visité.
Vacances d'été dans une autre ville	Ville inhabituelle, retraits hors domicile.	Séquence cohérente sur plusieurs jours, commerces de vacances.
Gros achat ponctuel (meubles, électronique, bijoux)	Montant supérieur à 3 000 TND.	Commerçant de la ville, carte à puce, en journée, aucun autre signal.
Nouveau téléphone	Nouvel appareil, parfois suivi d'un virement.	Enrôlement par OTP, pas d'échec de connexion, IP tunisienne.
Mot de passe oublié, changement de numéro en agence	Événements sensibles.	Canal agence, pas de virement anormal ensuite.
VPN, ordinateur partagé (bureau, cybercafé)	IP étrangère, appareil utilisé par plusieurs clients.	Historique stable du client, montants habituels.
Soirées de Ramadan, achats de l'Aïd	Opérations tardives, hausse des dépenses.	Calendrier (§ 5.8).

5.8 Le calendrier 2026 à connaître

Période (dates indicatives)	Effet sur les données
18 février → 19 mars : Ramadan	Activité tardive (jusqu'à 1 h du matin), hausse des courses alimentaires, Omra.
10 → 19 mars : fin du Ramadan	Achats de vêtements pour l'Aïd.

Période (dates indicatives)	Effet sur les données
20–21 mars : Aïd el-Fitr / 27–28 mai : Aïd al-Adha	Activité réduite.
18 → 26 mai : avant l'Aïd al-Adha	Retraits d'espèces importants (achat du mouton).
Du 25 au 3 de chaque mois	Jour de paie : plus de transactions.
Juillet → août	Voyages et vacances d'été (à l'étranger et en Tunisie).
1er → 20 septembre	Rentrée scolaire : éducation, habillement.
Week-end	Plus d'activité le samedi, moins le dimanche.

6. Trois parcours recommandés

Choisissez votre niveau d'ambition en fonction de votre équipe et du temps disponible. Chaque niveau est une solution **complète et démontrable**. Commencez par le Bronze, puis montez en gamme.

	Bronze : solide	Argent : convaincant	Or : expert
Données	transactions + labels + customers	+ cards, merchants, alertes du moteur de règles	+ digital_events, analyse des points communs entre victimes
Features	8 à 10 features comportementales (montant vs habitudes, nouveauté, vélocité, heure)	20 à 30 features : géographie, canal, plafonds, profil	+ features digitales et de graphe (commerçant, DAB, appareil communs)
Détection	Score heuristique ou modèle simple (régression logistique, arbre)	Gradient boosting, validation temporelle, comparaison chiffrée au moteur de règles	+ contributions SHAP par transaction, seuil choisi selon la capacité des analystes
Copilote	Top facteurs injectés dans le prompt, questions sur une transaction	Function calling : historique client, comparaison aux habitudes, file d'alertes	Multi-outils : relier des cas entre eux, rédiger le résumé d'investigation, expliquer les faux positifs
Interface	File d'alertes, fiche transaction, chat	+ vue client 360 (timeline, carte), décision	+ graphe des points communs, journal des décisions et feedback

> **Un Bronze parfaitement démontré vaut mieux qu'un Or inachevé.** Branchez le copilote le plus tôt possible sur un score simple, puis améliorez le score.

7. Feature engineering : par où commencer

Les données brutes ne suffisent pas : une transaction n'est suspecte que **par rapport aux habitudes** du client, de la carte ou de l'appareil. Chaque feature doit pouvoir se traduire en une phrase d'explication : c'est ce qui rendra votre copilote convaincant.

> **Règle d'or : uniquement le passé**
>
> Une feature calculée pour une transaction ne doit utiliser que les transactions et événements **antérieurs**. Calculer « la moyenne des montants du client » sur toute la période revient à regarder dans le futur : le score paraît excellent en test, puis s'effondre en réalité. Le jury vérifiera le code des équipes classées.

```
tx = tx.sort_values(["customer_id", "timestamp"])
g = tx.groupby("customer_id")["amount_tnd"]
# moyenne des montants AVANT la transaction (shift(1) exclut la transaction courante)
tx["avg_amount_before"] = g.transform(lambda s: s.shift(1).expanding().mean())
tx["amount_ratio"] = tx["amount_tnd"] / tx["avg_amount_before"]
# premier passage du client dans ce pays
tx["is_new_country"] = ~tx.duplicated(["customer_id", "country"])
```

7.1 Catalogue d'idées

Famille	Exemples de features	Phrase d'explication associée
Montant	Ratio à la moyenne ou à la médiane passée du client, z-score, montant / revenu mensuel, montant / plafond de la carte	« Montant 5 fois supérieur à la moyenne habituelle du client (151 TND). »
Temporalité	Heure, nuit, jour de la semaine, jour de paie, Ramadan / Aïd, temps écoulé depuis la transaction précédente	« Opération à 3 h du matin alors que le client n'opère jamais la nuit. »
Vélocité	Nombre et somme des transactions sur 10 min, 1 h, 24 h (par carte, client, appareil), refus récents	« 7 tentatives en 25 minutes, dont 4 refusées. »
Nouveauté	Nouveau pays, ville, commerçant, catégorie, appareil, bénéficiaire, pays IP	« Premier paiement physique hors de Tunisie de tout l'historique du client. »
Géographie	Distance au domicile, vitesse de déplacement depuis la transaction physique précédente	« Paiement à Shenzhen 2 heures après un retrait à Kairouan : déplacement impossible. »
Canal et sécurité	Mode d'entrée (piste magnétique, sans contact, sans 3DS), motif de refus, carte non internationale	« Piste magnétique utilisée alors que la carte est à puce. »
Banque digitale	Échecs de connexion sur 24 h, appareil enrôlé récemment, mot de passe réinitialisé, numéro modifié, bénéficiaire ajouté juste avant	« Bénéficiaire ajouté 4 minutes avant le virement, depuis un appareil enrôlé le jour même. »
Profil	Âge, segment, revenu, ancienneté, niveau KYC, type de carte	« Client retraité avec une carte nationale : un achat en ligne à l'étranger est atypique. »
Réseau (graphe)	Commerçants ou DAB communs à des cartes déjà fraudées, appareils partagés entre clients	« 12 autres cartes passées par ce DAB ont été fraudées dans les 3 semaines. »

Astuce pour les features de graphe : ne comptez que les fraudes **déjà signalées** à la date de la transaction (`fraud_reported_date` antérieure à `timestamp`). C'est le seul usage légitime de cette colonne comme feature.

Dans Fabric, les fenêtres glissantes se calculent efficacement avec les fonctions `Window` de PySpark (exemple en annexe C).

8. Détection et scoring

8.1 Commencez par un score heuristique

En une heure, un score à points (nouveau pays +3, montant supérieur à 3 fois la moyenne +2, rafale de transactions +3…) vous donne une première file d'alertes, une base de comparaison et un plan B. Il est aussi très facile à expliquer.

8.2 Passez au supervisé

- **Régression logistique** : lisible, coefficients directement interprétables.
- **Gradient boosting** (HistGradientBoosting, LightGBM, XGBoost) : en général le plus performant sur ce type de données tabulaires.
- Gérez le déséquilibre des classes avec `class_weight`, `scale_pos_weight` ou un sous-échantillonnage des transactions légitimes.

- Une approche **hybride** (modèle + quelques règles métier explicites) est tout à fait valable si elle est justifiée.

8.3 Validez dans le temps, jamais au hasard

Entraînez sur janvier → mai et validez sur juin → juillet. Un découpage aléatoire mélange des transactions d'un même épisode de fraude entre entraînement et validation et surestime fortement la performance.

8.4 Les bonnes métriques

Métrique	Pourquoi
Accuracy	**À éviter** : un modèle qui ne détecte rien a déjà 99,3 % d'accuracy.
PR-AUC (average precision)	Métrique principale, adaptée aux classes très déséquilibrées.
Précision et rappel sur le top 1 %	Les analystes ne peuvent traiter qu'un volume limité d'alertes (~1 000 sur deux mois). Parmi elles, combien de vraies fraudes ? Quelle part des fraudes est couverte ?
Comparaison au moteur de règles	Précision de ~15 % et rappel de ~42 % sur la période d'entraînement : votre référence.
Rappel par typologie	Montre où votre modèle est fort, et où il l'est moins (honnêteté appréciée).

8.5 Rendre le score explicable

Pour chaque transaction alertée, calculez et stockez ses **3 à 5 facteurs principaux** dans une table (par exemple `gold_alert_explanations`) : nom du facteur, valeur observée, valeur habituelle, contribution. Sources possibles : valeurs SHAP, contributions de la régression logistique, ou règles déclenchées. C'est ce que le copilote lira pour expliquer, au lieu d'inventer.

8.6 Les pièges à éviter

Piège	Pourquoi c'est un problème	Correction
Features calculées sur toute la période	Fuite d'information future, score artificiellement élevé	Fenêtres passées, `shift(1)`, `expanding()`
`fraud_scenario` ou `fraud_reported_date` comme feature directe	Ce sont des labels, inconnus au moment de la transaction	Analyse et cible uniquement (voir l'astuce graphe du § 7.1)
Découpage aléatoire train / validation	Surestimation de la performance	Découpage temporel
Optimiser l'accuracy	Classes déséquilibrées	PR-AUC, top 1 %
Utiliser l'ordre des identifiants	Aucune signification métier	Ne pas utiliser les ID comme features numériques
Laisser le LLM calculer le score	Non reproductible, risque d'hallucination	Le modèle calcule, le LLM explique
Envoyer des milliers de lignes dans le prompt	Coût, latence, limite de contexte	Des outils qui renvoient des agrégats

9. Concevoir le copilote

9.1 Le principe : ancrer chaque réponse dans les données

Le score est calculé par votre modèle. Le LLM ne fait que **chercher les faits** grâce à des outils, puis les **traduire** en une explication métier. Il ne doit rien affirmer que ses outils ne lui ont pas renvoyé.

Question de l'analyste	**Azure OpenAI** choisit un outil	**Outil** interroge les tables Gold	**Résultat** faits chiffrés (JSON)	**Réponse** expliquée et sourcée

9.2 Les outils (fonctions) recommandés

Outil	Ce qu'il renvoie
`get_alert_queue(date, n)`	Les transactions les plus risquées d'une journée, avec score et facteurs principaux.
`get_transaction(transaction_id)`	Le détail de la transaction, son score et ses facteurs de risque.
`get_customer_profile(customer_id)`	Le profil et les habitudes : montant moyen, pays, villes, horaires, appareils, bénéficiaires.
`compare_to_habits(transaction_id)`	Les écarts chiffrés entre la transaction et l'historique du client.
`get_customer_history(customer_id, days)`	Les transactions et événements récents du client.
`get_digital_events(customer_id, around)`	Connexions, appareils, bénéficiaires et changements de numéro autour d'une opération.
`find_related_cases(transaction_id)`	D'autres cartes ou clients partageant le même commerçant, DAB, appareil ou bénéficiaire.
`record_decision(transaction_id, decision, comment)`	Enregistre la décision de l'analyste (bloquer / escalader / valider) et son commentaire.

9.3 Prompt système conseillé

```
Tu es le copilote de l'analyste fraude de la Banque. Tu réponds en français,
de façon concise, factuelle et structurée.
Règles :
- N'affirme que ce que les outils t'ont renvoyé. Cite montants, dates et identifiants.
- Si une information manque, dis-le et propose la vérification à faire.
- Ne calcule jamais toi-même un score : utilise celui fourni par le modèle.
- Distingue les faits (données) de ton interprétation.
- Si l'opération semble légitime, explique aussi pourquoi.
- Termine par une recommandation : BLOQUER, ESCALADER ou VALIDER, avec un niveau
  de confiance. La décision finale appartient à l'analyste.
```

9.4 Format de réponse recommandé

1. Verdict	Niveau de risque et score du modèle.
2. Facteurs principaux	Trois au maximum, chacun chiffré et comparé aux habitudes du client.
3. Contexte	Profil du client, activité récente, événements digitaux utiles.
4. Éléments rassurants	Ce qui modère le risque (voyage prévu, appareil connu…).
5. Recommandation	BLOQUER / ESCALADER / VALIDER et niveau de confiance.

9.5 Exemple 1 : une fraude (données réelles du jeu d'entraînement)

Analyste : « Pourquoi la transaction TX00267400 est-elle suspecte ? »

Risque élevé (score du modèle : 0,94, valeur illustrative).

Facteurs principaux

- Paiement physique à **Shenzhen (Chine)** : les 155 paiements et retraits physiques précédents du client ont tous eu lieu en Tunisie.
- Montant de **783,611 TND**, soit **5,2 fois** sa moyenne (151 TND), au-dessus de 95 % de ses opérations passées.
- **13 minutes plus tôt**, retrait de 800 TND à un DAB de Shenzhen avec la **piste magnétique**, typique d'une carte copiée.

Contexte : client de 37 ans, segment Particulier, domicilié à Kairouan ; dernière opération en Tunisie la veille à 17 h 01 ; aucun billet d'avion ni hôtel réservé.

Recommandation : **BLOQUER** la carte CARD001261 et contacter le client (confiance élevée).

Mise à jour à 15 h 55 : la même carte est utilisée à un DAB de Kairouan. Le client ne peut pas être aux deux endroits : le clonage se confirme.

9.6 Exemple 2 : un faux positif évité

Analyste : « Le moteur de règles a levé une alerte sur TX00283105 (3 086 TND à Istanbul). Faut-il bloquer ? »

Risque faible : l'alerte est très probablement injustifiée.

Éléments rassurants

- Billets d'avion achetés en ligne les 19 et 24 juin.
- Aucune opération physique en Tunisie depuis le 30 juin au soir : le client est en voyage, et ses paiements à Istanbul s'enchaînent de façon cohérente depuis le 1er juillet.
- Paiement par **puce**, pas par piste magnétique.

Point d'attention : montant inhabituel (environ 2 fois son 95e centile) dans une bijouterie, mais compatible avec un client Professionnel (revenu de 4 893 TND).

Recommandation : **VALIDER**, éventuellement avec une notification au client (confiance moyenne à élevée).

À noter : ce voyage a déclenché **21 alertes** du moteur de règles actuel, toutes des faux positifs.

9.7 Questions que le jury pourra poser en direct

- « Pourquoi cette transaction est-elle suspecte ? » / « Quels sont les principaux facteurs de risque ? »
- « Montre-moi les 10 transactions les plus risquées d'hier. »
- « Cette transaction ressemble-t-elle au comportement habituel du client ? »
- « Résume pourquoi cette alerte doit être priorisée. »
- « Que s'est-il passé dans l'application juste avant ce virement ? »
- « D'autres clients sont-ils passés par le même commerçant ou le même DAB ? »
- « Pourquoi cette alerte n'est-elle **pas** une fraude ? »
- « Rédige le résumé d'investigation pour le dossier. »
- « Combien d'alertes inutiles évites-tu par rapport au moteur de règles actuel ? »

La quantité de questions supportées compte moins que la **qualité des réponses** : reliées aux données, utiles et directement exploitables.

9.8 Garde-fous

- Température basse (0 à 0,2) pour des réponses stables et factuelles.
- Gestion propre des cas inconnus (identifiant inexistant, question hors sujet).

- Journalisation des questions, des outils appelés et des réponses : la traçabilité est une exigence réglementaire.
- La décision reste humaine : le copilote recommande, l'analyste décide.
- Ne jamais versionner les clés Azure OpenAI dans GitHub : utilisez des variables d'environnement ou un fichier `.env` ignoré.

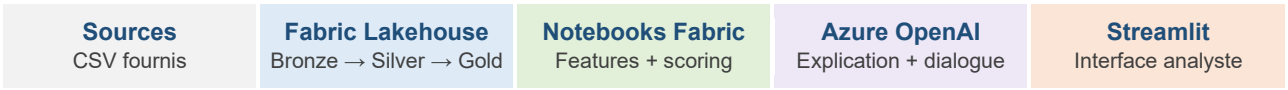
10. L'interface analyste

Écran	Contenu	Conseil
File d'alertes	Transactions classées par score, filtres (date, canal, typologie, niveau de risque)	Code couleur par niveau de risque, nombre d'alertes du jour
Fiche transaction	Détail, score, 3 facteurs principaux, boutons de décision	C'est le moment fort de la démo : tout doit être visible sans défiler
Vue client 360	Profil, habitudes, timeline des transactions et événements digitaux, carte des lieux	`st.map` avec latitude et longitude, mise en évidence de l'opération alertée
Copilote	Zone de chat, questions suggérées en boutons	Passer automatiquement la transaction ouverte comme contexte
Décision et feedback	Bloquer / escalader / valider + commentaire, journal des décisions	Illustre la boucle d'amélioration continue
Tableau de bord (optionnel)	Volume d'alertes, comparaison avec le moteur de règles	Un graphique suffit

Pour une démo fluide

- Pré-calculez scores et explications dans Fabric : n'entraînez jamais le modèle dans l'application.
- Chargez des fichiers Parquet ou une table Gold filtrée, et utilisez `st.cache_data`.
- Préparez 3 cas de démo : une fraude évidente, une fraude subtile, un faux positif évité.
- Prévoyez une capture vidéo de secours en cas de problème réseau.

11. Architecture de référence sur Microsoft Fabric



En transverse : **GitHub + GitHub Copilot** pour le code (notebooks, application, prompts, tests).

Couche	Où	Contenu suggéré
Bronze	Lakehouse (Files + tables Delta)	Les CSV chargés tels quels : `bronze_transactions_train`, `bronze_customers`…
Silver	Tables Delta	Données typées, nettoyées et jointes : `silver_transactions` (train + test + client + carte + commerçant).
Gold	Tables Delta	`gold_features`, `gold_scores` (score + facteurs), `gold_customer_profile`, `gold_alert_queue`.
Scoring	Notebook Fabric (PySpark / pandas, suivi MLflow possible)	Entraînement, validation temporelle, scoring d'août à septembre.
Application	Streamlit + Azure OpenAI	Lit les tables Gold, appelle le LLM avec des outils.

Relier Streamlit aux données Fabric

- **Option A** : interroger les tables Gold via le point de terminaison SQL (SQL analytics endpoint) du Lakehouse, avec authentification Microsoft Entra.
- **Option B (plan B)** : exporter les tables Gold en Parquet depuis Fabric et les lire localement. Plus simple en cas de difficulté d'accès, à condition de montrer clairement le pipeline Fabric en amont.

Pourquoi Fabric est central

- Il réunit ingestion, stockage, transformation et exploitation dans un environnement unifié.
- Il rend le projet plus industrialisable qu'un script local.
- Il renforce la cohérence de la solution dans l'écosystème Microsoft.

12. Organisation et planning conseillés

Phase	Part du temps	Jalon de fin de phase
1. Cadrage et ingestion	10 %	Données chargées dans Fabric (Bronze), rôles répartis, parcours choisi.
2. Exploration et features	20 %	Table de features (Gold) et 2 ou 3 constats métier à raconter.
3. Scoring	20 %	Score validé sur juin → juillet, comparaison au moteur de règles.
4. Explication et copilote	25 %	Le copilote répond aux 4 premières questions du § 9.7.
5. Interface et intégration	15 %	Démo de bout en bout fonctionnelle.
6. Pitch et répétition	10 %	Pitch chronométré, cas de démo figés, vidéo de secours.

Rôles suggérés (cumulables)

Rôle	Responsabilités
Data engineer	Lakehouse Fabric, tables Bronze / Silver / Gold, export pour l'application.
Data scientist	Features, modèle, validation, facteurs explicatifs.
AI engineer	Outils du copilote, prompt système, garde-fous, tests de questions.
Développeur front / UX	Application Streamlit, parcours de démo.
Product owner / pitch	Vision métier, cas de démo, storytelling, chronométrage.

> **Conseil** : branchez dès que possible le copilote et l'interface sur un premier score heuristique, puis remplacez-le par le modèle. Une équipe qui intègre tout à la dernière heure prend un gros risque.

13. Livrables

Livrable	Contenu attendu
Application fonctionnelle	File d'alertes ou transactions, score ou logique de priorisation explicite, navigation simple et démontrable en quelques minutes.
Copilote (obligatoire)	Zone de dialogue, questions libres ou semi-guidées, réponses contextualisées et explications claires.
Démonstration	Cas réel (issu des données) ou simulé, interaction live pendant le pitch, justification claire de la suspicion.

Livrable	Contenu attendu
Restitution technique	Architecture, rôle de Fabric dans le pipeline, choix du modèle ou de la logique de scoring, performance en validation.
Usage de GitHub Copilot	Exemples concrets : composants générés, tests, refactorisation, temps gagné.
Dépôt GitHub	Code, notebooks Fabric exportés, `requirements.txt`, README expliquant comment lancer la solution.
Fichier de scores (optionnel)	Voir § 14.

14. Soumission des scores (optionnelle)

Pour un classement technique complémentaire, déposez un fichier CSV contenant **une ligne par transaction** de `transactions_test.csv` :

```
transaction_id,fraud_score,is_alert
TX00333060,0.0213,0
TX00333061,0.9731,1
```

- `fraud_score` (obligatoire) : score continu, plus élevé = plus suspect.
- `is_alert` (facultatif) : votre décision d'alerte (0/1) selon votre propre seuil.

Le jury calculera la PR-AUC (métrique principale) et la précision / le rappel sur le top 1 %. Ce classement est un **bonus** : il ne remplace pas la grille du § 15. Le code des équipes les mieux classées sera relu (absence de fuite d'information future).

15. Critères d'évaluation

Critère	Ce que regarde le jury	Poids indicatif
Pertinence métier	La solution répond au besoin réel de l'analyste : priorisation, décision, réduction des faux positifs.	20 %
Qualité de la détection	Démarche, validation temporelle, comparaison au moteur de règles, absence de fuite d'information.	20 %
Explication et copilote	Réponses ancrées dans les données, chiffrées, utiles, y compris sur les cas légitimes.	25 %
Expérience et démo	Fluidité, clarté, storytelling, interaction live.	15 %
Cohérence technologique	Rôles clairs de Fabric, Azure OpenAI et GitHub Copilot, architecture lisible.	15 %
Restitution	Qualité du pitch et des réponses aux questions du jury.	5 %

Ce qui n'est pas acceptable

- Un dashboard sans intelligence ni interaction.
- Un scoring sans explication.
- Des réponses génériques de chatbot, non reliées aux données du cas.
- Un projet sans interface démontrable.

16. Règles du jeu

1. Les données fournies sont synthétiques et suffisent. Si vous utilisez des données externes publiques, citez-les.

2. Aucune information postérieure à une transaction ne doit servir à la scorer (voir § 7 et § 8.6).
3. Les labels du jeu de test ne sont pas communiqués : la validation se fait sur juin → juillet.
4. Les outils d'IA sont autorisés et encouragés, à commencer par GitHub Copilot. Indiquez comment vous les avez utilisés.
5. Aucune clé, aucun mot de passe et aucun secret dans le dépôt GitHub.
6. Le jury peut demander à consulter le code, les notebooks et l'historique Git de toute équipe.

17. Réussir son pitch

Format conseillé : **5 minutes**, démo live au centre.

Temps	Contenu
0:00 – 0:45	Le problème, chiffré : 85 % de faux positifs, 58 % des fraudes manquées par les règles actuelles.
0:45 – 3:00	Démo live : une alerte → la fiche transaction → une question au copilote → la décision. Puis un faux positif évité.
3:00 – 4:00	Sous le capot : pipeline Fabric, modèle, résultats de validation comparés au moteur de règles.
4:00 – 4:30	Comment GitHub Copilot vous a fait gagner du temps.
4:30 – 5:00	Limites assumées et prochaines étapes.

> Le chemin le plus convaincant reste : **une transaction → un score → une explication → une décision**. Il doit être visible par le jury en quelques secondes.

18. Message final

Ce challenge vous invite à concevoir plus qu'un modèle : un **assistant bancaire intelligent** qui aide réellement l'analyste fraude à détecter, comprendre et décider. En combinant **Microsoft Fabric** pour la plateforme data, **Azure OpenAI** pour les explications et **GitHub Copilot** pour accélérer le développement, vous pouvez produire une démonstration moderne, crédible et à forte valeur métier.

La meilleure proposition sera celle qui transformera des données complexes en décisions exploitables, avec une expérience simple, claire et convaincante. En d'autres termes : **moins de bruit, plus d'explication, plus de valeur pour l'analyste.**

Bon hackathon !

Annexe A – Glossaire

Terme	Définition
TPE	Terminal de paiement électronique : paiement par carte en magasin.
DAB	Distributeur automatique de billets.
3-D Secure (3DS)	Authentification forte du porteur lors d'un paiement en ligne (code reçu par SMS ou validation dans l'application).
Piste magnétique	Ancienne technologie de lecture de carte, facile à copier, contrairement à la puce.
MCC	Merchant Category Code : code qui décrit l'activité du commerçant.
OTP	One-Time Password : code à usage unique, souvent envoyé par SMS.
SIM swap	Le fraudeur fait transférer le numéro de téléphone de la victime sur sa propre carte SIM pour recevoir ses codes OTP.
Skimming	Copie des données d'une carte grâce à un dispositif posé sur un DAB ou un TPE.
Card testing	Test de numéros de cartes volés par de petits paiements, avant de les utiliser pour de gros montants.
Account takeover (ATO)	Prise de contrôle d'un compte de banque en ligne par un fraudeur.
Faux positif	Opération légitime signalée à tort comme suspecte.
PR-AUC	Aire sous la courbe précision-rappel : mesure la qualité d'un classement sur des classes déséquilibrées.
Fuite d'information (leakage)	Utilisation, pendant l'entraînement, d'informations qui ne seraient pas disponibles au moment de la prédiction.
Lakehouse	Stockage Fabric qui combine fichiers et tables Delta interrogeables en SQL et en Spark.
Bronze / Silver / Gold	Couches de données : brute, nettoyée et jointe, prête pour l'usage métier.
Function calling	Capacité du LLM à appeler des fonctions de votre code (vos outils) pour obtenir des données.

Annexe B – FAQ

Doit-on utiliser tous les fichiers ?
Non. Transactions, labels et clients suffisent pour une solution complète. Les autres fichiers sont des bonus.

Les données sont-elles réelles ?
Non, elles sont entièrement synthétiques. Elles reproduisent des comportements et des typologies de fraude réalistes.

Le taux de fraude est-il réaliste ?
Il est volontairement plus élevé que dans la réalité (environ 0,65 % contre environ 0,1 %) pour vous donner assez d'exemples.

Peut-on utiliser la colonne fraud_scenario ?
Oui, comme cible (classification multi-classes) ou pour analyser vos résultats, mais jamais comme feature.

Comment évaluer notre modèle sans les labels de test ?
Validez sur juin → juillet. Les labels d'août et septembre sont réservés au jury.

Peut-on utiliser un autre LLM qu'Azure OpenAI ?
Azure OpenAI est recommandé. Si vous en utilisez un autre, justifiez-le.

Peut-on utiliser Power BI ?
Oui, en complément, mais il ne remplace ni l'interface interactive ni le copilote.

Que faire si Fabric ou Azure OpenAI n’est pas accessible ?
Prévenez immédiatement les organisateurs. En attendant, avancez en local (pandas, Parquet) et basculez dès que possible.

Annexe C – Code de démarrage

C.1 Charger les fichiers dans le Lakehouse (notebook Fabric, PySpark)

Déposez d’abord les CSV dans `Files/hackathon/` du Lakehouse.

```
files = ["customers", "cards", "merchants", "transactions_train", "transactions_test",
         "fraud_labels_train", "legacy_rule_alerts_train", "digital_events"]
for f in files:
    df = (spark.read.option("header", True).option("inferSchema", True)
          .csv(f"Files/hackathon/{f}.csv"))
    df.write.mode("overwrite").format("delta").saveAsTable(f"bronze_{f}")
```

C.2 Une feature de vélocité (PySpark, passé uniquement)

```
from pyspark.sql import functions as F, Window
tx = spark.table("bronze_transactions_train") \
    .withColumn("ts", F.col("timestamp").cast("timestamp").cast("long"))
# transactions de la même carte dans l’heure précédente (transaction courante exclue)
w1h = Window.partitionBy("card_id").orderBy("ts").rangeBetween(-3600, -1)
tx = tx.withColumn("card_tx_last_1h", F.count("*").over(w1h))
```

C.3 Appeler Azure OpenAI avec un outil (Python)

```
import os, json
from openai import AzureOpenAI
client = AzureOpenAI(api_key=os.environ["AZURE_OPENAI_KEY"],
                     azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
                     api_version="2024-10-21")
tools = [{"type": "function", "function": {
    "name": "get_transaction",
    "description": "Détail, score et facteurs de risque d’une transaction",
    "parameters": {"type": "object",
                   "properties": {"transaction_id": {"type": "string"}},
                   "required": ["transaction_id"]}}}]
messages = [{"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": "Pourquoi TX00267400 est-elle suspecte ?"}]
resp = client.chat.completions.create(model="<nom-du-deploiement>", messages=messages,
                                      tools=tools, temperature=0.1)
# si resp demande un outil : exécuter la fonction, ajouter son résultat à messages,
# puis rappeler l’API pour obtenir la réponse finale
```

C.4 Squelette d’interface Streamlit

```
import pandas as pd
import streamlit as st
st.set_page_config(page_title="Copilote Fraude", layout="wide")

@st.cache_data
def load_alerts():
    return pd.read_parquet("gold_alert_queue.parquet")

alerts = load_alerts()
tx_id = st.sidebar.selectbox("Alerte", alerts["transaction_id"])
st.dataframe(alerts[alerts.transaction_id == tx_id])

history = st.session_state.setdefault("history", [])
for m in history:
    st.chat_message(m["role"]).write(m["content"])
if question := st.chat_input("Posez votre question sur cette alerte…"):
    answer = ask_copilot(question, context_tx=tx_id)   # votre fonction Azure OpenAI
    history += [{"role": "user", "content": question},
                {"role": "assistant", "content": answer}]
    st.rerun()
```

Fin du document.